Guia de Defesa Técnica e Apresentação - Gabriel Lira Figueira

Projeto: Estrutura de Dados S.A. (Bubble Sort Decrescente) | **Disciplina:** Estrutura de Dados (Unit)

Aluno (Você):	**Gabriel Lira Figueira**	**Responsabilidade:**	BubbleSortParalelo.java, Arquitetura (Strategy), Main.java, TestesBubbleSort.java
Integrante 2:	João Luís Aragão Rocha	**Responsabilidade:**	ArquivoServico.java, BubbleSortClassico.java
Integrante 3:	Eduarda Santana Santos	**Responsabilidade:**	Registro.java, Ordenador.java, JavaDocs

1. BubbleSortParalelo.java: O Motor Concorrente (Transposição Par-Ímpar)

Como funciona: Para eliminar a *Condição de Corrida (Race Condition)* sem locks pesados, o vetor é ordenado em duas fases alternadas com pares disjuntos: (1) **Fase Ímpar:** threads processam simultaneamente pares iniciados em ímpares (1,2), (3,4), (5,6)... (2) **Fase Par:** threads processam pares iniciados em pares (0,1), (2,3), (4,5)... Como os pares não se tocam, as threads rodam em paralelo real aproveitando todos os núcleos.

```
// Detecção automática de núcleos da CPU (sem fixar valor arbitrário)
int cores = Math.max(1, Runtime.getRuntime().availableProcessors());

// Fatiamento (Chunking) e ajuste fino de paridade para cada Thread:
int inicioReal = inicioThread;
if ((inicioReal % 2) != (inicioParidade % 2)) inicioReal++;

// Barreira de Sincronização Nativa:
for (Thread t : threads) t.join(); // Fase seguinte só inicia após término da atual

// Fallback Sequencial para lotes < 500 itens (elimina overhead de criação de threads)
```

2. Arquitetura de Software: Padrão de Projeto Strategy (POO & DRY)

Você estabeleceu o contrato através da interface Ordenador. Isso isola o algoritmo de ordenação do resto do sistema (Princípio Open/Closed do SOLID). No Main.java, alternar entre o Bubble Sort clássico e o paralelo custa apenas **uma única linha de código**, sem precisar mexer em leitura, gravação ou métricas.

3. Main.java: Orquestração Industrial, Telemetria e CLI

Implementou argumentos dinâmicos via terminal (args[0] e args[1]), telemetria com System.currentTimeMillis() isolando tempos de I/O de Leitura, Ordenação de CPU e I/O de Escrita, e o método exibirAmostraConsole() que imprime o Top 5 empresas mais valiosas e a menor no rodapé.

4. TestesBubbleSort.java: Suíte com 21 Asserções Automatizadas

```
[CENÁRIO 1] Ordenação Decrescente Estrita: Topo = 9000.0, Base = 1000.0 (Clássico e Paralelo idênticos)
[CENÁRIO 2] Casos de Borda (Corner Cases): Vetor vazio (sem crash), 1 elemento intacto, empates de valor mantidos
[CENÁRIO 3] Melhor e Pior Caso: Vetor já ordenado (parada antecipada ativa) e vetor em ordem inversa
[CENÁRIO 4] I/O Físico: Criação, leitura, ordenação e gravação real de arquivo TXT com limpeza automática
[CENÁRIO 5] Teste de Carga: 5.000 empresas aleatórias geradas e ordenadas em tempo real (100% integridade)
```

5. Respostas Prontas para Perguntas do Professor

P1: Por que você usou threads no Bubble Sort?

R: O enunciado fala em 1 milhão de registros. Como o Bubble Sort é $O(N^2)$, uma thread única demoraria dias. A Transposição Par-Ímpar é a formulação acadêmica oficial para paralelizar Bubble Sort dividindo pares disjuntos entre os núcleos da máquina.

P2: Como o código lida com empate de valor entre empresas?

R: O algoritmo é estável. A troca usa estritamente menor que (<). Havendo valores iguais, não há troca desnecessária, preservando a ordem original de inserção.

P3: Por que vetor primitivo Registro[] e não ArrayList?

R: Para eliminar o overhead de chamadas virtuais (.get() e .set()) e reduzir indireções na Heap. O array contíguo permite que a CPU aproveite a memória Cache L1/L2 nos swaps constantes.